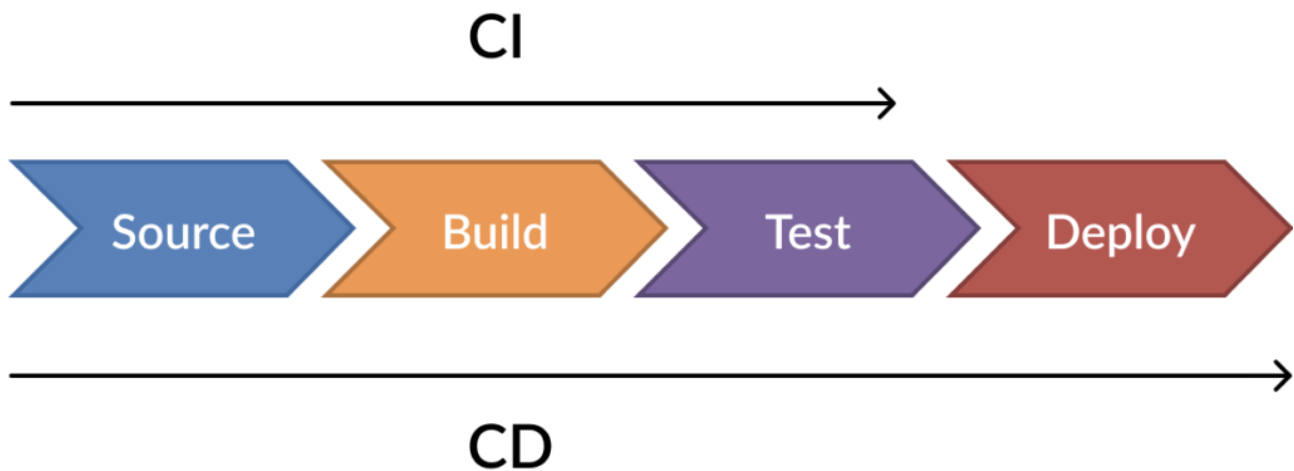


Exp11: Create a CI/CD pipeline to automate build, test, and deployment of a Docker application



What we do

- Create a simple app
- Push to GitHub
- Automatically:
 - ✓ Build Docker image
 - ✓ Run test
 - ✓ Run container

Step 1: Check Prerequisites

I. Verify Git Installation

```
git --version
```

```
cse@cse-B760M-H:~$ git --version
git version 2.43.0
cse@cse-B760M-H:~$
```

Install if not installed:

```
sudo apt update
```

```
sudo apt install git -y
```

```
git --version
```

II. Verify Docker Installation

```
docker --version
```

```
cse@cse-B760M-H:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
cse@cse-B760M-H:~$
```

III. Verify Docker Service

Check whether Docker daemon is running:

```
sudo systemctl status docker
```

You should see:

```
active (running)
```

```
cse@cse-B760M-H:~$ sudo systemctl status docker
[sudo] password for cse:
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-05-09 22:52:04 IST; 5h 16min left
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2227 (dockerd)
      Tasks: 50
     Memory: 103.9M (peak: 113.1M)
        CPU: 1.009s
     CGroup: /system.slice/docker.service
             └─2227 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.672624703+05:30" level=warning msg="Error (Unable to complete atomic operation, key modified) deleting object [end
May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.772916304+05:30" level=info msg="Loading containers: done."
May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.806883666+05:30" level=info msg="Docker daemon" commit="28.2.2-0ubuntu1~24.04.1" containerd-snapshotter=false stor
May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.807164854+05:30" level=info msg="Initializing buildkit"
May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.808178019+05:30" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or dire
May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.818587446+05:30" level=info msg="Completed buildkit initialization"
May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.824933612+05:30" level=info msg="Daemon has completed initialization"
May 09 22:52:04 cse-B760M-H dockerd[2227]: time="2026-05-09T22:52:04.825082475+05:30" level=info msg="API listen on /run/docker.sock"
May 09 22:52:04 cse-B760M-H systemd[1]: Started docker.service - Docker Application Container Engine.
lines 1-22/22 (END)
```

Install if not installed:

```
sudo apt update
```

```
sudo apt install docker.io -y
```

Start Docker:

```
sudo systemctl start docker
```

Enable Docker on boot:

```
sudo systemctl enable docker
```

Verify Docker Without sudo (Recommended)

Run:

```
docker run hello-world
```

```
cse@cse-B760M-H:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

cse@cse-B760M-H:~$
```

If permission denied occurs then:

Add current user to Docker group:

```
sudo usermod -aG docker $USER
```

Then logout and login again.

Verify:

```
docker run hello-world
```

IV. Verify Internet Connectivity

GitHub Actions requires internet.

Check connectivity:

```
ping github.com
```

OR

```
ping google.com
```

Press:

CTRL + C

to stop.

```
cse@cse-B760M-H:~$ ping github.com
PING github.com (20.207.73.82) 56(84) bytes of data.
64 bytes from 20.207.73.82: icmp_seq=1 ttl=115 time=17.7 ms
64 bytes from 20.207.73.82: icmp_seq=2 ttl=115 time=17.6 ms
64 bytes from 20.207.73.82: icmp_seq=3 ttl=115 time=17.6 ms
64 bytes from 20.207.73.82: icmp_seq=4 ttl=115 time=17.6 ms
^C
--- github.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 17.597/17.636/17.697/0.037 ms
cse@cse-B760M-H:~$ ping google.com
PING google.com (172.217.24.110) 56(84) bytes of data.
64 bytes from lcmaaa-ap-in-f14.1e100.net (172.217.24.110): icmp_seq=1 ttl=114 time=7.17 ms
64 bytes from lcmaaa-ap-in-f14.1e100.net (172.217.24.110): icmp_seq=2 ttl=114 time=7.02 ms
64 bytes from lcmaaa-ap-in-f14.1e100.net (172.217.24.110): icmp_seq=3 ttl=114 time=7.01 ms
64 bytes from lcmaaa-ap-in-f14.1e100.net (172.217.24.110): icmp_seq=4 ttl=114 time=7.02 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 7.008/7.055/7.169/0.065 ms
cse@cse-B760M-H:~$
```

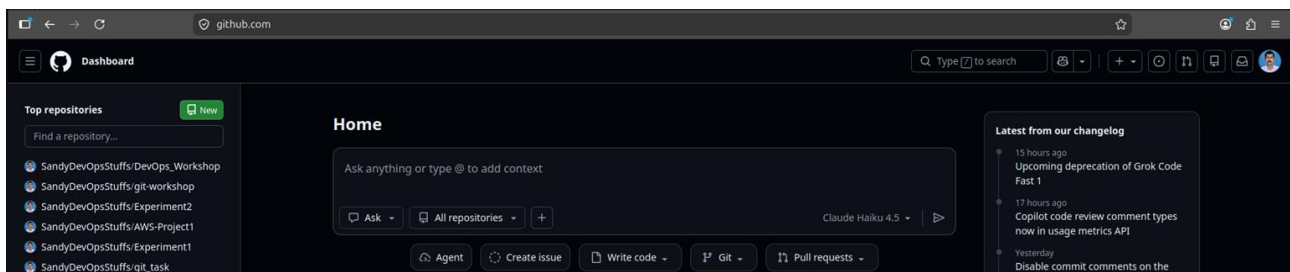
V. Verify GitHub account access

Open browser and login:

<https://github.com>

Ensure:

- ✓ account accessible
- ✓ repository creation works



VI. Configure Git Username and Email (VERY IMPORTANT)

Check current config:

```
git config --global user.name
```

```
git config --global user.email
```

```
cse@cse-B760M-H:~$ git config --global user.name
cse@cse-B760M-H:~$ git config --global user.email
cse@cse-B760M-H:~$
```

If not configured then:

Set username:

```
git config --global user.name "<Your Name>"
```

Ex:

```
git config --global user.name "Sandesh D Manocharya"
```

Verify:

```
git config --global user.name
```

```
cse@cse-B760M-H:~$ git config --global user.name "Sandesh D Manocharya"
cse@cse-B760M-H:~$ git config --global user.name
Sandesh D Manocharya
cse@cse-B760M-H:~$
```

Set email:

```
git config --global user.email "sandy.devops.stuffs@gmail.com"
```

```
git config --global user.email
```

```
cse@cse-B760M-H:~$ git config --global user.email "sandy.devops.stuffs@gmail.com"
cse@cse-B760M-H:~$ git config --global user.email
sandy.devops.stuffs@gmail.com
cse@cse-B760M-H:~$
```

Verify:

```
git config --list
```

```
cse@cse-B760M-H:~$ git config --list
user.name=Sandesh D Manocharya
user.email=sandy.devops.stuffs@gmail.com
cse@cse-B760M-H:~$
```

This is:

- ✓ NOT login authentication
- ✓ NOT GitHub password

It is only commit author identity. Git uses this name in commits.

Example commit history:

Author: Sandesh D Manocharya

VII. Verify GitHub Authentication

GitHub password is no longer used for Git push operations.

Instead, GitHub uses Personal Access Tokens for secure authentication.

Recommended method: Personal Access Token (PAT)

When pushing:

- ✓ Username = GitHub username
- ✓ Password = PAT token

Generate and configure the token

Give a name: CT_LAB_TOKEN

Expiration:

Choose:

30 days

OR

No expiration

(for lab systems)

Select Scopes:

Check:

- ✓ Repo
- ✓ workflow

This is enough for pushing repositories and for GitHub Actions (CICD).

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

CT_LAB_TOKEN

What's this token for?

Expiration

📅 30 days (Jun 08, 2026) ▼

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

- | | |
|---|--------------------------------------|
| ☒ repo | Full control of private repositories |
| ☒ repo:status | Access commit status |
| ☒ repo_deployment | Access deployment status |
| ☒ public_repo | Access public repositories |
| ☒ repo:invite | Access repository invitations |
| ☒ security_events | Read and write security events |
| ☒ workflow | Update GitHub Action workflows |

Step 2: Create Project Folder

```
mkdir cicd-demo
```

```
cd cicd-demo
```

```
cse@cse-B760M-H:~$ mkdir cicd-demo
cse@cse-B760M-H:~$ cd cicd-demo
cse@cse-B760M-H:~/cicd-demo$
```

Step 3: Create Simple Python App

```
nano app.py
```

Paste the following:

```
print("Hello from CI/CD Pipeline")
```

Save and exit the nano editor

Step 4: Create Dockerfile

```
nano Dockerfile
```

Paste the following:

```
FROM python:3.9-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

Step 5: (Optional but Recommended) Add Test

```
nano test.sh
```

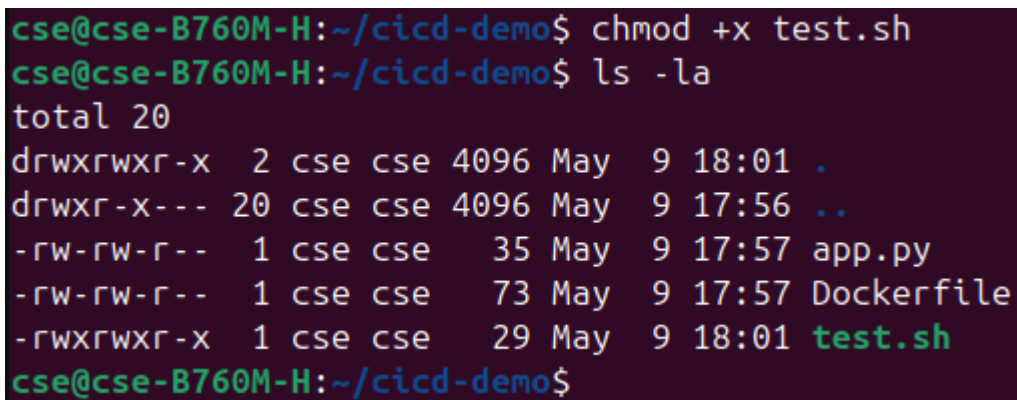
Paste the following:

```
python app.py | grep "Hello"
```

Make executable:

```
chmod +x test.sh
```

```
ls -la
```

A terminal window screenshot with a dark purple background. The prompt is 'cse@cse-B760M-H:~/cicd-demo\$'. The first command is 'chmod +x test.sh'. The second command is 'ls -la', which outputs a directory listing for the current directory. The listing shows permissions, file size, owner, group, and date for the current directory, parent directory, and files 'app.py', 'Dockerfile', and 'test.sh'. The 'test.sh' file is highlighted in green in the original image.

```
cse@cse-B760M-H:~/cicd-demo$ chmod +x test.sh
cse@cse-B760M-H:~/cicd-demo$ ls -la
total 20
drwxrwxr-x  2 cse cse 4096 May  9 18:01 .
drwxr-x--- 20 cse cse 4096 May  9 17:56 ..
-rw-rw-r--  1 cse cse   35 May  9 17:57 app.py
-rw-rw-r--  1 cse cse   73 May  9 17:57 Dockerfile
-rwxrwxr-x  1 cse cse   29 May  9 18:01 test.sh
cse@cse-B760M-H:~/cicd-demo$
```

Step 5: Push to GitHub

```
git init
```



```
cse@cse-B760M-H:~/cicd-demo$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/cse/cicd-demo/.git/
cse@cse-B760M-H:~/cicd-demo$
```

ls -la

```
cse@cse-B760M-H:~/cicd-demo$ ls -la
total 24
drwxrwxr-x  3 cse cse 4096 May  9 18:03 .
drwxr-x--- 20 cse cse 4096 May  9 17:56 ..
-rw-rw-r--  1 cse cse   35 May  9 17:57 app.py
-rw-rw-r--  1 cse cse   73 May  9 17:57 Dockerfile
drwxrwxr-x  7 cse cse 4096 May  9 18:03 .git
-rwxrwxr-x  1 cse cse   29 May  9 18:01 test.sh
cse@cse-B760M-H:~/cicd-demo$
```

git add .

git commit -m "Initial commit"

```
cse@cse-B760M-H:~/cicd-demo$ git add .
cse@cse-B760M-H:~/cicd-demo$ git commit -m "Initial commit"
[master (root-commit) 2f27b89] Initial commit
 3 files changed, 6 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 app.py
 create mode 100755 test.sh
cse@cse-B760M-H:~/cicd-demo$
```

Create repo in GitHub

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

 SandyDevOpsStuffs ▾

Repository name *

/ ccd-demo

✓ ccd-demo is available.

Great repository names are short and memorable. How about [vigilant-broccoli?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

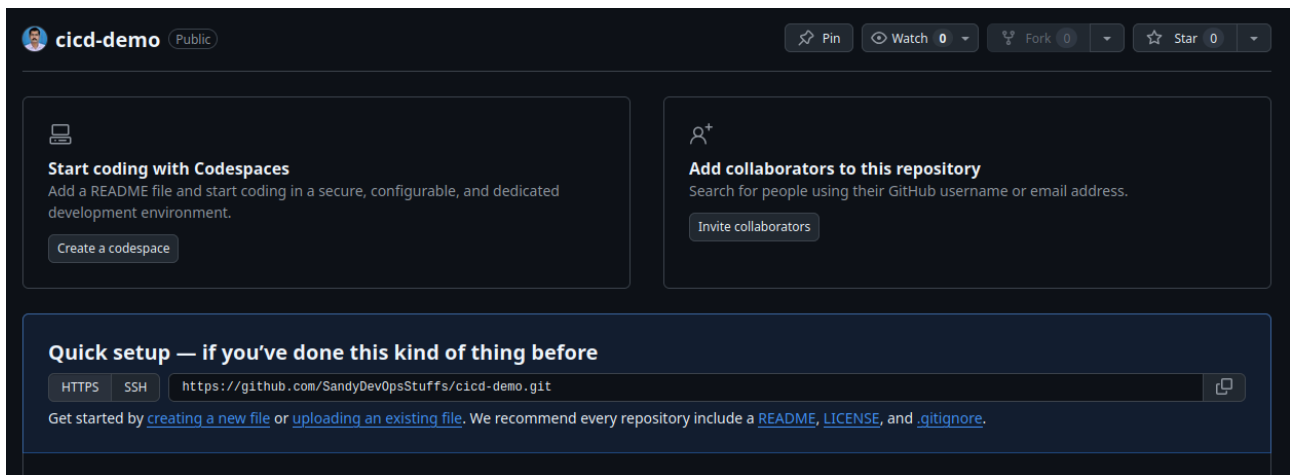
No license ▾

Connect GitHub Apps

Connect this repository to apps SandyDevOpsStuffs is subscribed to

Choose apps

Create repository



Then:

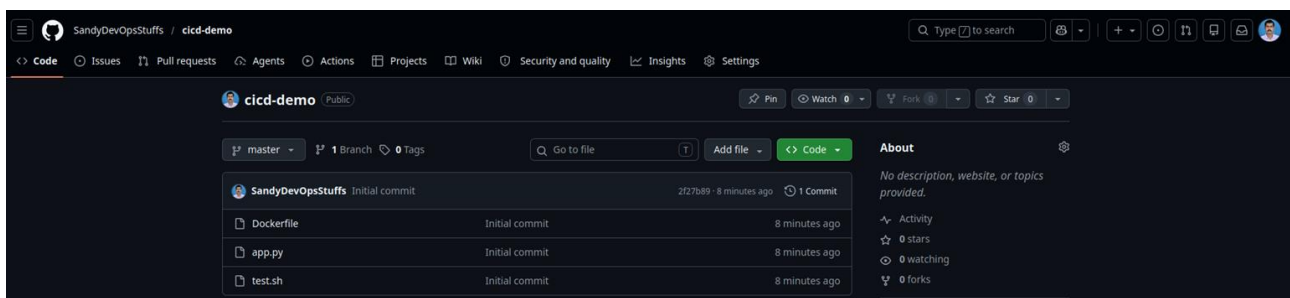
```
git remote add origin https://github.com/SandyDevOpsStuffs/cicd-demo.git
```

```
git push -u origin master
```

When it asks for GitHub credentials, you enter GitHub user name and for password you enter PAT instead of password of GitHub.

```
cse@cse-B760M-H:~/cicd-demo$ git remote add origin https://github.com/SandyDevOpsStuffs/cicd-demo.git
cse@cse-B760M-H:~/cicd-demo$ git push -u origin master
Username for 'https://github.com': SandyDevOpsStuffs
Password for 'https://SandyDevOpsStuffs@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 28 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 448 bytes | 448.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/SandyDevOpsStuffs/cicd-demo.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
cse@cse-B760M-H:~/cicd-demo$
```

Verify whether all files are pushed to remote repo



Step 6: Create GitHub Actions Workflow

```
ls -la
```

```
cse@cse-B760M-H:~/cicd-demo$ ls -la
total 24
drwxrwxr-x  3 cse cse 4096 May  9 18:03 .
drwxr-x--- 20 cse cse 4096 May  9 17:56 ..
-rw-rw-r--  1 cse cse   35 May  9 17:57 app.py
-rw-rw-r--  1 cse cse   73 May  9 17:57 Dockerfile
drwxrwxr-x  8 cse cse 4096 May  9 18:11 .git
-rwxrwxr-x  1 cse cse   29 May  9 18:01 test.sh
cse@cse-B760M-H:~/cicd-demo$
```

```
mkdir -p .github/workflows
```

```
ls -la
```

```
cse@cse-B760M-H:~/cicd-demo$ mkdir -p .github/workflows
cse@cse-B760M-H:~/cicd-demo$
cse@cse-B760M-H:~/cicd-demo$
cse@cse-B760M-H:~/cicd-demo$ ls -la
total 28
drwxrwxr-x  4 cse cse 4096 May  9 18:14 .
drwxr-x--- 20 cse cse 4096 May  9 17:56 ..
-rw-rw-r--  1 cse cse   35 May  9 17:57 app.py
-rw-rw-r--  1 cse cse   73 May  9 17:57 Dockerfile
drwxrwxr-x  8 cse cse 4096 May  9 18:11 .git
drwxrwxr-x  3 cse cse 4096 May  9 18:14 .github
-rwxrwxr-x  1 cse cse   29 May  9 18:01 test.sh
cse@cse-B760M-H:~/cicd-demo$
```

```
nano .github/workflows/main.yml
```

Paste This YAML

```
name: CI/CD Pipeline
on:
  push:
    branches:
      - master

jobs:
  build-test-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up Docker
        run: docker --version

      - name: Build Docker Image
        run: docker build -t myapp .

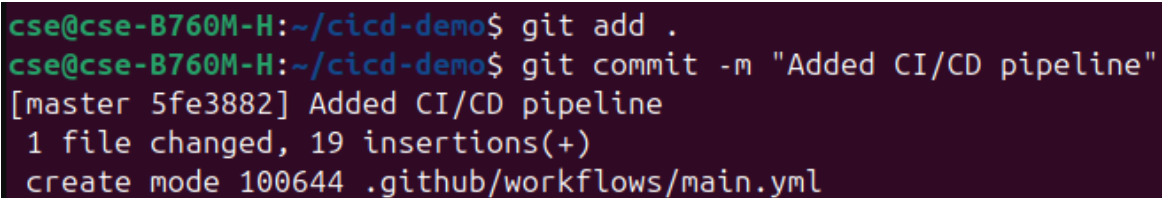
      - name: Run Test
        run: docker run myapp

      - name: Run Container
        run: docker run myapp
```

Step 7: Push Workflow

```
git add .
```

```
git commit -m "Added CI/CD pipeline"
```

A terminal window with a dark background and light-colored text. It shows the execution of two git commands: 'git add .' and 'git commit -m "Added CI/CD pipeline"'. The output of the second command is displayed, showing the commit hash [master 5fe3882], the commit message 'Added CI/CD pipeline', the number of files changed (1), the number of insertions (19), and the file mode (100644) for .github/workflows/main.yml.

```
cse@cse-B760M-H:~/cicd-demo$ git add .
cse@cse-B760M-H:~/cicd-demo$ git commit -m "Added CI/CD pipeline"
[master 5fe3882] Added CI/CD pipeline
1 file changed, 19 insertions(+)
create mode 100644 .github/workflows/main.yml
```

```
git push
```

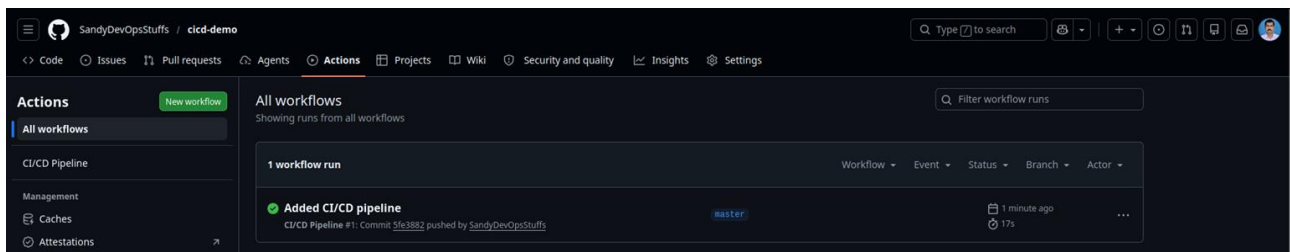
```
cse@cse-B760M-H:~/cicd-demo$ git push
Username for 'https://github.com': SandyDevOpsStuffs
Password for 'https://SandyDevOpsStuffs@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 28 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 670 bytes | 670.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/SandyDevOpsStuffs/cicd-demo.git
    2f27b89..5fe3882  master -> master
cse@cse-B760M-H:~/cicd-demo$
```

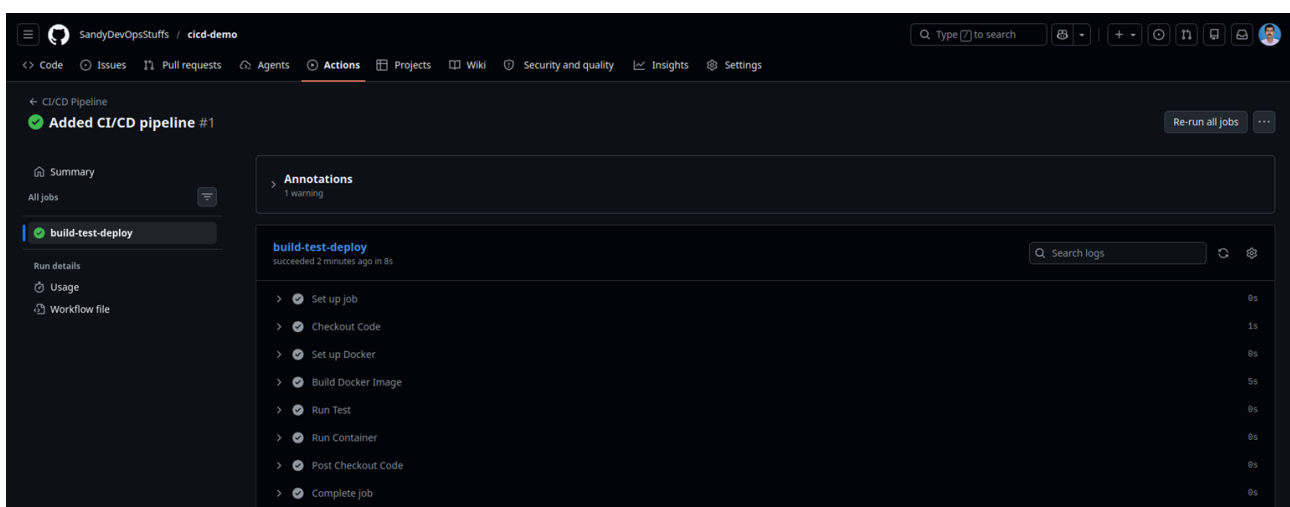
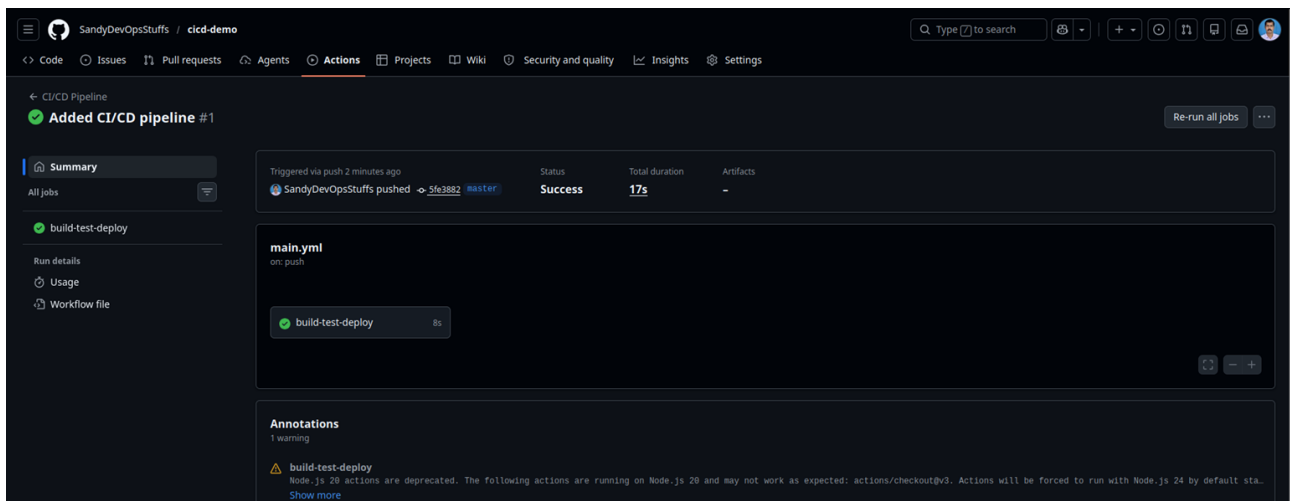
Step 8: Verify Pipeline

Go to GitHub → **Actions** tab

You will see:

- ✓ Workflow running
- ✓ Build step
- ✓ Test step
- ✓ Output logs





Expected Output:

- ✓ Pipeline runs automatically on push
- ✓ Docker image builds
- ✓ App prints:

Hello from CI/CD Pipeline

Step 9: Once again verify CICD pipeline:

Make a small change in `app.py`

```
print("Welcome to automated CI/CD Pipeline")
```

Modify `test.sh` accordingly.

```
python app.py | grep "Welcome"
```

```
git add .
```

```
git commit -m "Updated application message"
```

```
git push
```

```
cse@cse-B760M-H:~/cicd-demo$ git add .
cse@cse-B760M-H:~/cicd-demo$ git commit -m "Updated application message"
[master e2c35f2] Updated application message
 2 files changed, 2 insertions(+), 2 deletions(-)
cse@cse-B760M-H:~/cicd-demo$ git push
Username for 'https://github.com': SandyDevOpsStuffs
Password for 'https://SandyDevOpsStuffs@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 28 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 458 bytes | 458.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/SandyDevOpsStuffs/cicd-demo.git
   5fe3882..e2c35f2  master -> master
cse@cse-B760M-H:~/cicd-demo$
```

Now go back to GitHub and observe the changes in new Workflow Run (*new commit message*).

